



Compiler

Segment 1

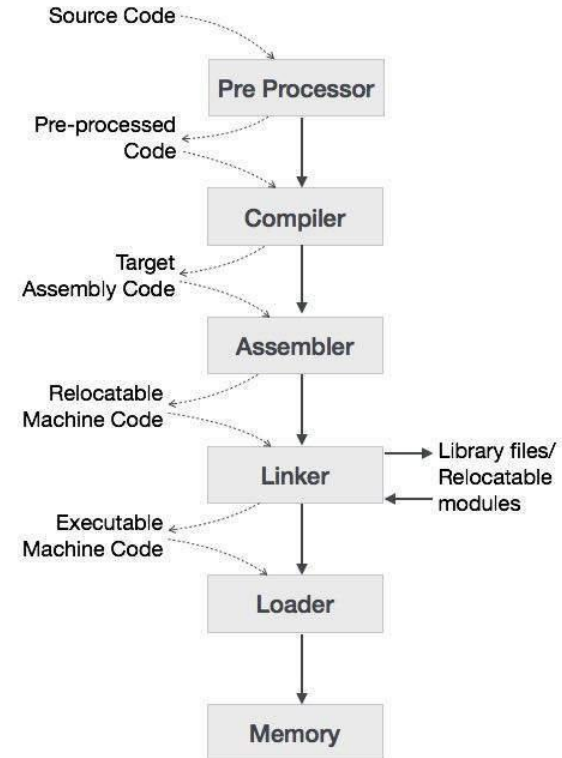
Md. Nazmul Arefin

Lecturer

Dept. of Computer Science & Engineering
International Islamic University Chittagong

A language processing system

- We write programs in high-level language, which is easier for us to understand and remember.
- These programs are then fed into a series of tools and OS components to get the desired code that can be used by the machine. This is known as Language Processing System.
- The high-level language is converted into binary language in various phases.
- A compiler is a program that converts high-level language to assembly language.
- Similarly, an assembler is a program that converts the assembly language to machine-level language.



A language processing system



Let us first understand how a program, using C compiler, is executed on a host machine.

- User writes a program in C language (high-level language).
- The C compiler, compiles the program and translates it to assembly program (low-level language).
- An assembler then translates the assembly program into machine code (object).
- A linker tool is used to link all the parts of the program together for execution (executable machine code).
- A loader loads all of them into memory and then the program is executed.

A language processing system (tools)



Preprocessor

A preprocessor, generally considered as a part of compiler, is a tool that produces input for compilers. It deals with macro-processing, augmentation, file inclusion, language extension, etc.

Assembler

An assembler translates assembly language programs into machine code. The output of an assembler is called an object file, which contains a combination of machine instructions as well as the data required to place these instructions in memory.

Linker

Linker is a computer program that links and merges various object files together in order to make an executable file. All these files might have been compiled by separate assemblers. The major task of a linker is to search and locate referenced module/routines in a program and to determine the memory location where these codes will be loaded, making the program instruction to have absolute references.

A language processing system (tools)



Loader

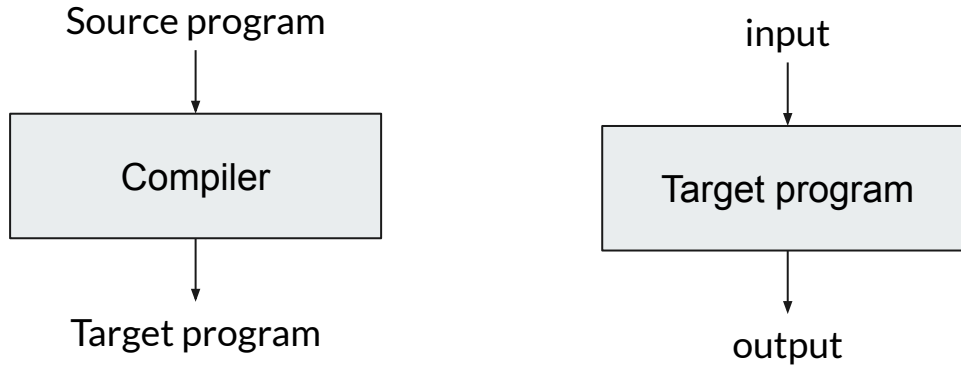
Loader is a part of operating system and is responsible for loading executable files into memory and execute them. It calculates the size of a program (instructions and data) and creates memory space for it. It initializes various registers to initiate execution.

Cross-compiler

A compiler that runs on platform (A) and is capable of generating executable code for platform (B) is called a cross-compiler.

Compiler

- A program that can read a program in one language (Source language) and translate it into an equivalent program in another language (target language).
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



interpreter

- Instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on inputs supplied by the user.



Benefits and difference

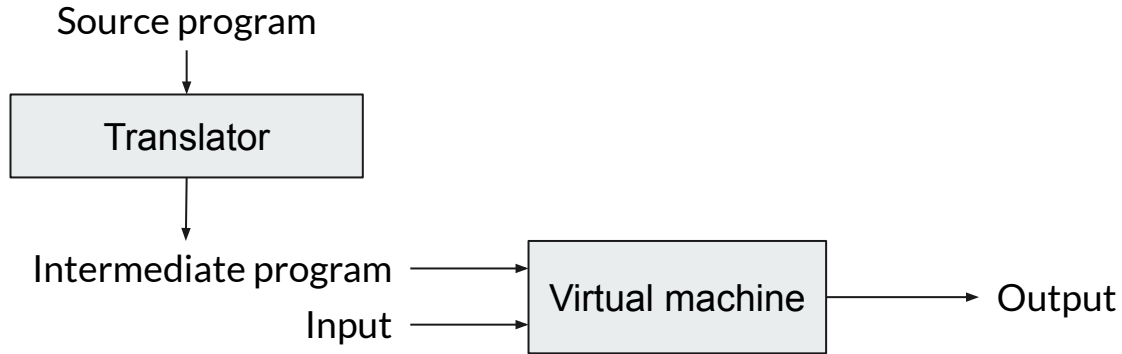


- The machine language target program produced by a compiler is usually much faster
- Interpreter can give better error diagnostics than a compiler, because it executes the source program statement by statement.

Compiler	Interpreter
Compiler scans the whole program in one go.	Translates program one statement at a time.
As it scans the code in one go, the errors (if any) are shown at the end together.	Considering it scans code one line at a time, errors are shown line by line.
C, C++, C# etc.	Python, Ruby, Perl, SNOBOL, MATLAB etc.

Example

- Java language processors combine compilation and interpretation.
- Intermediate code: *bytecode*





Compiler Design - Architecture

Architecture



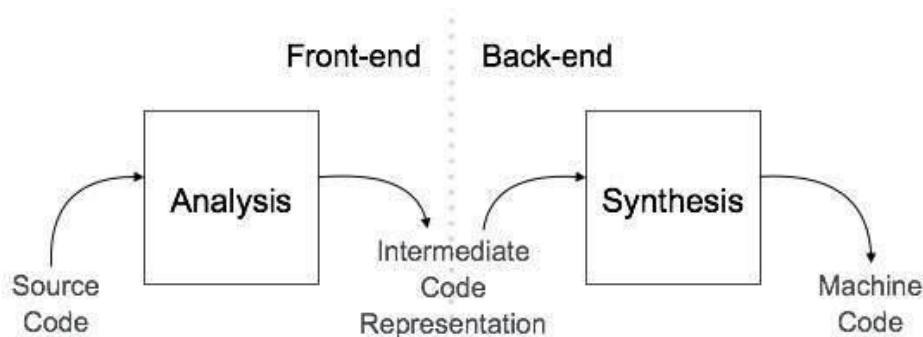
A compiler can broadly be divided into two phases based on the way they compile.

1. Analysis phase
2. Synthesis phase

Architecture

Analysis Phase

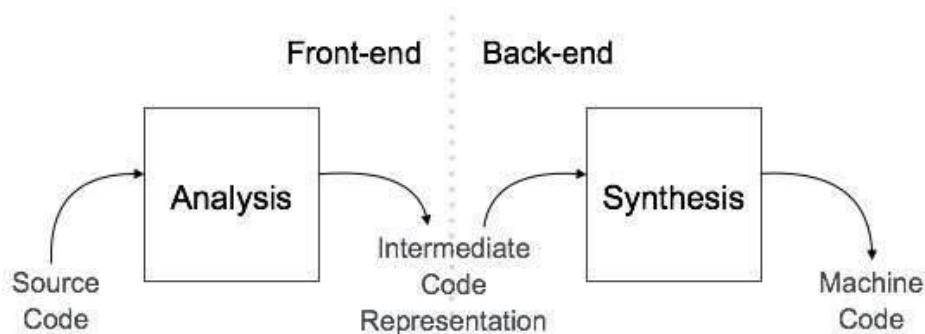
- Known as the front-end of the compiler
- The analysis phase of the compiler reads the source program, divides it into core parts and then checks for lexical, grammar and syntax errors.
- The analysis phase generates an intermediate representation of the source program and symbol table, which should be fed to the Synthesis phase as input.



Architecture

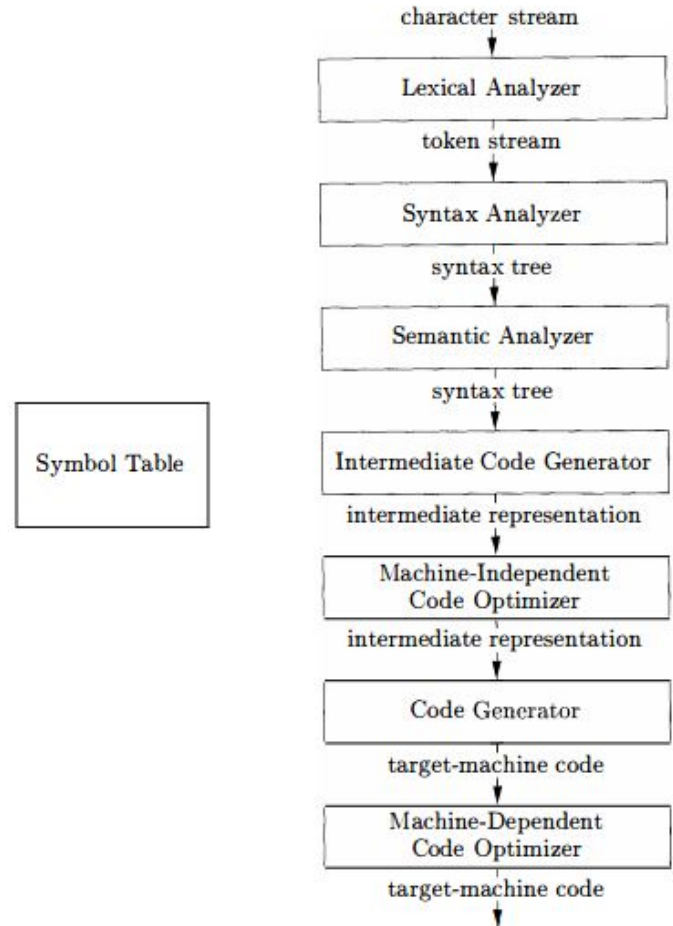
Synthesis Phase

- Known as the back-end of the compiler, the synthesis phase generates the target program with the help of intermediate source code representation and symbol table.



Phases of Compiler

- The compilation process is a sequence of various phases.
- Each phase takes input from its previous stage, has its own representation of source program, and feeds its output to the next phase of the compiler.



Lexical Analysis



The first phase of a compiler is called *lexical analysis* or *scanning*.

The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called lexemes.

For each lexeme, the lexical analyzer produces as output a token of the form that it passes on to the subsequent phase, syntax analysis.:

$\langle token-name, attribute-value \rangle$

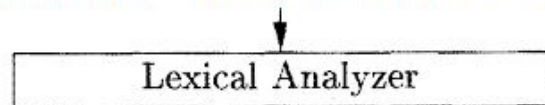
In the token, the first component token-name is an abstract symbol that is used during syntax analysis, and the second component attribute-value points to an entry in the symbol table for this token.

Information from the symbol-table entry is needed for semantic analysis and code generation.

Lexical Analysis

1. `position` is a lexeme that would be mapped into a token $\langle \text{id}, 1 \rangle$, where `id` is an abstract symbol standing for *identifier* and 1 points to the symbol-table entry for `position`. The symbol-table entry for an identifier holds information about the identifier, such as its name and type.
2. The assignment symbol `=` is a lexeme that is mapped into the token $\langle = \rangle$. Since this token needs no attribute-value, we have omitted the second component. We could have used any abstract symbol such as `assign` for the token-name, but for notational convenience we have chosen to use the lexeme itself as the name of the abstract symbol.
3. `initial` is a lexeme that is mapped into the token $\langle \text{id}, 2 \rangle$, where 2 points to the symbol-table entry for `initial`.
4. `+` is a lexeme that is mapped into the token $\langle + \rangle$.
5. `rate` is a lexeme that is mapped into the token $\langle \text{id}, 3 \rangle$, where 3 points to the symbol-table entry for `rate`.

`position = initial + rate * 60`



$\langle \text{id}, 1 \rangle \langle = \rangle \langle \text{id}, 2 \rangle \langle + \rangle \langle \text{id}, 3 \rangle \langle * \rangle \langle 60 \rangle$

Att. val.	token	name
1	id	position
2	id	initial
3	id	rate

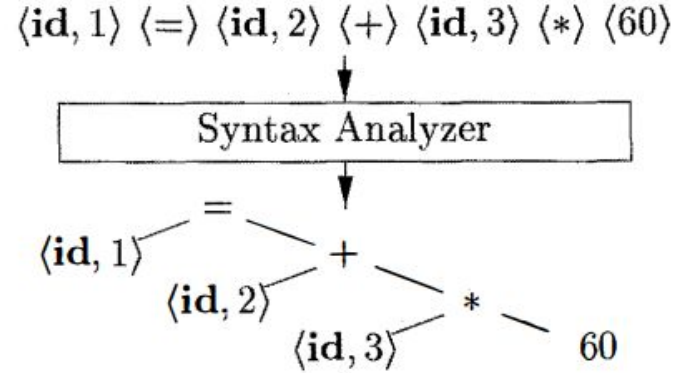
Symbol Table

Syntax Analysis

The second phase of the compiler is **syntax analysis** or **parsing**.

The parser uses the first components of the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.

A typical representation is a syntax tree in which each interior node represents an operation and the children of the node represent the arguments of the operation.



Syntax Analysis



This tree shows the order in which the operations in the assignment

`position = initial + rate * 60`

are to be performed. The tree has an interior node labeled `*` with `(id, 3)` as its left child and the integer `60` as its right child. The node `(id, 3)` represents the identifier `rate`. The node labeled `*` makes it explicit that we must first multiply the value of `rate` by `60`. The node labeled `+` indicates that we must add the result of this multiplication to the value of `initial`. The root of the tree, labeled `=`, indicates that we must store the result of this addition into the location for the identifier `position`.

This ordering of operations is consistent with the usual conventions of arithmetic which tell us that multiplication has higher precedence than addition, and hence that the multiplication is to be performed before the addition.

Semantic Analysis



The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.

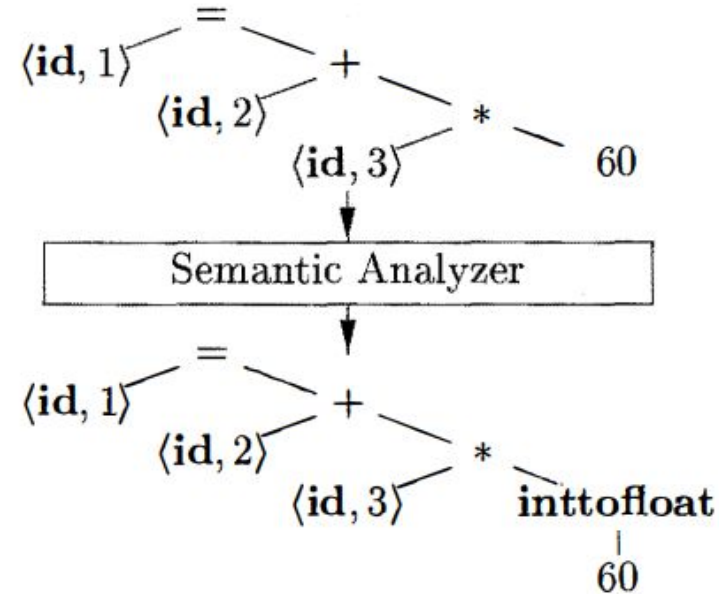
It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.

An important part of semantic analysis is **type checking**, where the compiler checks that each operator has matching operands.

For example, many programming language definitions require an array index to be an integer; the compiler must report an error if a floating-point number is used to index an array.

Semantic Analysis

Suppose that **position**, **initial**, and **rate** have been declared to be floating-point numbers, and that the lexeme 60 by itself forms an integer. The type checker in the semantic analyzer in the figure discovers that the operator ***** is applied to a floating-point number **rate** and an integer 60. In this case, the integer may be converted into a floating-point number.



Intermediate Code Generation

We consider an intermediate form called *three-address code*, which consists of a sequence of assembly-like instructions with three operands per instruction.

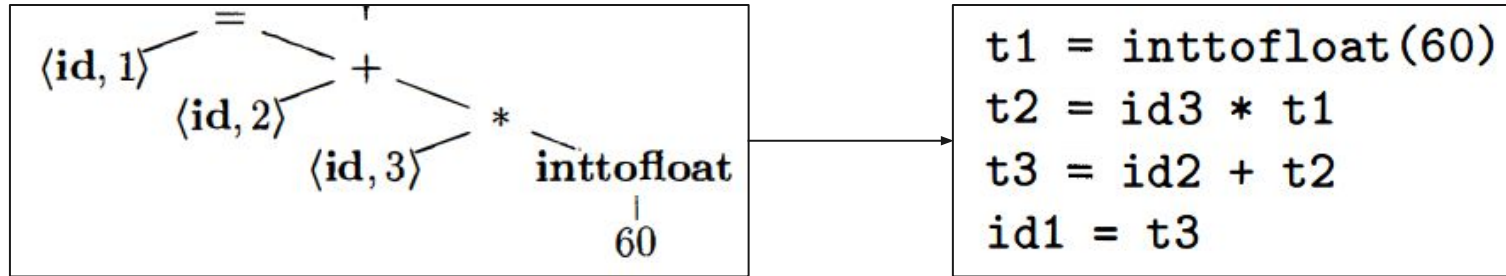


Fig. 1.3

Intermediate Code Generation



```
t1 = inttfloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

There are several points worth noting about three-address instructions.

First, each three-address assignment instruction has at most one operator on the right side. Thus, these instructions fix the order in which operations are to be done; the multiplication precedes the addition in the source program.

Second, the compiler must generate a temporary name to hold the value computed by a three-address instruction.

Third, some "three-address instructions" like the first and last in the sequence above, have fewer than three operands

Code Optimization



The machine-independent code-optimization phase attempts to improve the intermediate code so that better target code will result. Usually better means faster, but other objectives may be desired, such as shorter code, or target code that consumes less power.

The optimizer can deduce that the conversion of 60 from integer to floating point can be done once and for all at compile time, so the *inttofloat* operation can be eliminated by replacing the integer 60 by the floating-point number 60.0.

Moreover, t3 is used only once to transmit its value to id1 so the optimizer can transform (1.3) into the shorter sequence:

Code Optimization

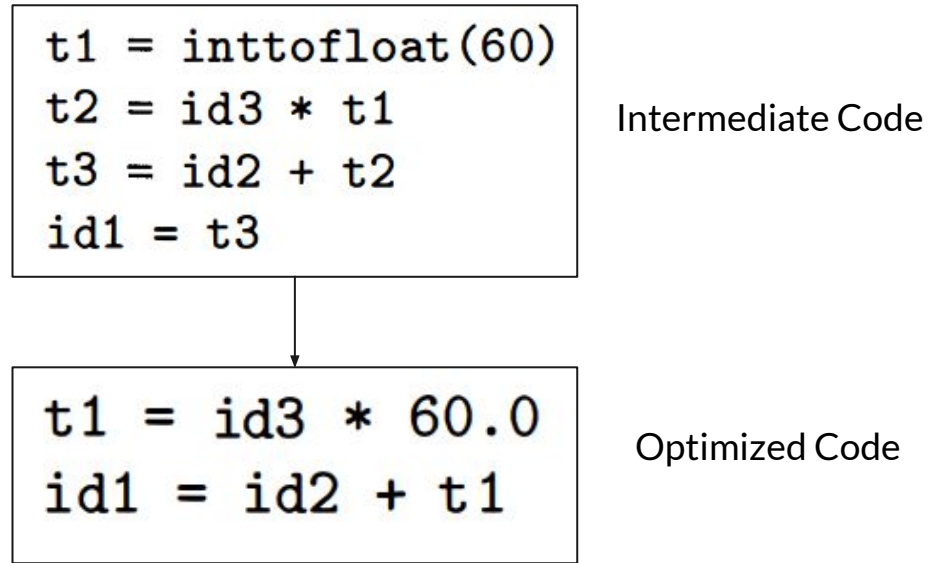


Fig. 1.4

Code Generation



The code generator takes as input an intermediate representation of the source program and maps it into the target language.

If the target language is machine code, registers Or memory locations are selected for each of the variables used by the program.

Then, the intermediate instructions are translated into sequences of machine instructions that perform the same task.

A crucial aspect of code generation is the judicious assignment of registers to hold variables.

Code Generation

For example, using registers R1 and R2, the intermediate code in (1.4) might get translated into the machine code

```
t1 = id3 * 60.0  
id1 = id2 + t1
```

```
LDF  R2,  id3  
MULF R2,  R2, #60.0  
LDF  R1,  id2  
ADDF R1,  R1, R2  
STF  id1, R1
```

Fig. 1.5

Code Generation

The F in each instruction tells us that it deals with floating-point numbers.

The contents of address id3 into register R2. , then multiplies it with floating-point constant 60.0. The # signifies that 60.0 is to be treated as an immediate constant.

The third instruction moves id2 into register R1 and the fourth adds to it the value previously computed in register R2.

Finally, the value in register R1 is stored into the address of id1 , so the code correctly implements the assignment statement

```
position = initial + rate * 60
```

```
LDF  R2,  id3
MULF R2,  R2, #60.0
LDF  R1,  id2
ADDF  R1,  R1, R2
STF  id1, R1
```



THANK YOU